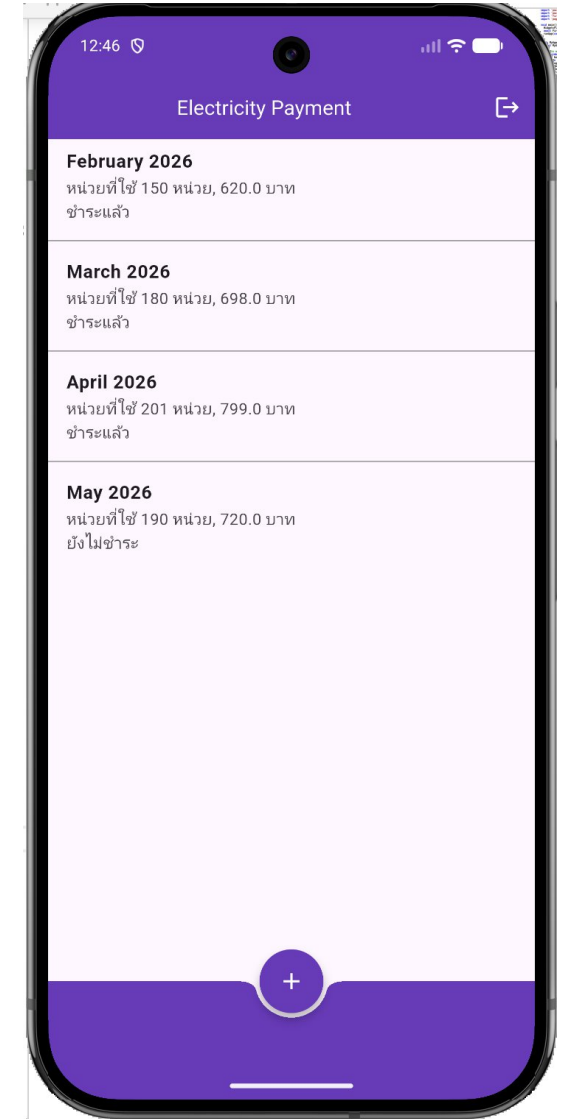
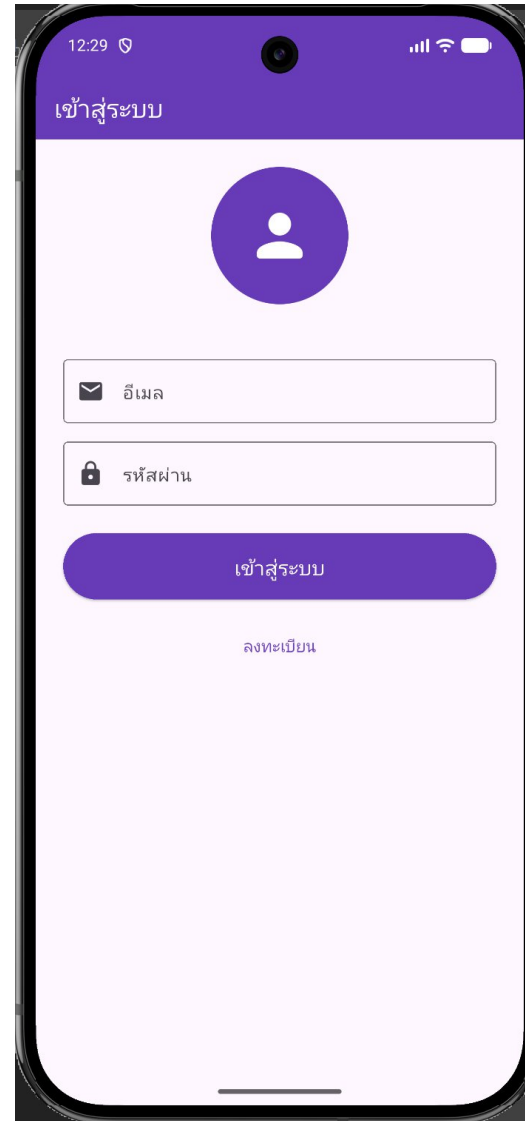


การตรวจสอบผู้ใช้งานด้วย Firebase Authentication

By Prasertsak U., Ph.D

Note

- เนื้อหาต่อจากนี้จะใช้ Flutter Project จาก Lab: Electricity Payment เป็นฐานในการพัฒนาเพิ่มเติม
- ความสามารถของระบบจะประกอบด้วย
 - การ Login โดยใช้ Email/Password ที่ลงทะเบียนไว้กับ Firebase Authentication
 - การลงทะเบียน Email/Password เพิ่มเติม



เตรียมพื้นฐานโปรเจกต์

- ให้สร้างโปรเจกต์ใหม่ชื่อว่า `electricity_payment_auth`
- นำพื้นฐานโปรแกรมตั้งต้นจาก [ที่นี่](#) โดยหลังจาก Download มาแล้วให้แตกไฟล์ .ZIP และนำไฟล์ทั้งหมดมาวางทับภายในโปรเจกต์ใหม่ที่เราสร้างขึ้น
- หลังจากนั้นให้เปิดโปรเจกต์ด้วย VS-Code และเปิด Terminal ภายใน VS-Code และรันคำสั่ง `firebase login` ให้เรียบร้อย
- หลังจาก login เรียบร้อยแล้ว ให้รันคำสั่ง `flutterfire configure` โดยให้เลือก Firebase Project ที่ใช้กับ Lab ก่อนหน้า และเลือก Platform เป็น `android` และ `web`

เตรียมพื้นฐานโปรเจกต์ (cont.)

- แก้ไขเพิ่มเติมไฟล์ local.properties

```
local.properties ×
android > local.properties
1  sdk.dir=/Users/tinygrain/Library/Android/sdk
2  flutter.sdk=/Users/tinygrain/Development/flutterSDK
3  flutter.buildMode=debug
4  flutter.versionName=1.0.0
5  flutter.versionCode=1
6  flutter.ndkVersion=27.0.12077973
7  flutter.minSdkVersion=23
```

- เพิ่มเติมเนื้อหาภายในไฟล์ `android/app/build.gradle.kts`

```
1  plugins {
2      id("com.android.application")
3      id("kotlin-android")
4      // The Flutter Gradle Plugin must be applied
5      id("dev.flutter.flutter-gradle-plugin")
6      id("com.google.gms.google-services")
7  }
```

เพิ่มบรรทัดที่ 6

Flutter plugin for Firebase authentication

- For Dart below 2.15, please use `firebase_auth: ^3.3.8`

firebase_auth 6.4.0 

Published 22 days ago • firebase.google.com

SDK | FLUTTER | PLATFORM | ANDROID | IOS | MACOS | WEB | WINDOWS

[Readme](#) | [Changelog](#) | [Example](#) | [Installing](#) | [Versions](#)



Firebase Auth for Flutter

pubspec.yaml

```
30 dependencies:
31   flutter:
32     sdk: flutter
33
34   # The following adds the Cupertino Icons font
35   # Use with the CupertinoIcons class for iOS s
36   cupertino_icons: ^1.0.8
37   cloud_firestore: ^6.3.0
38   firebase_core: ^4.7.0
39   intl: ^0.20.2
40   month_picker_dialog: ^6.7.2
41   firebase_auth: ^6.4.0
42
```



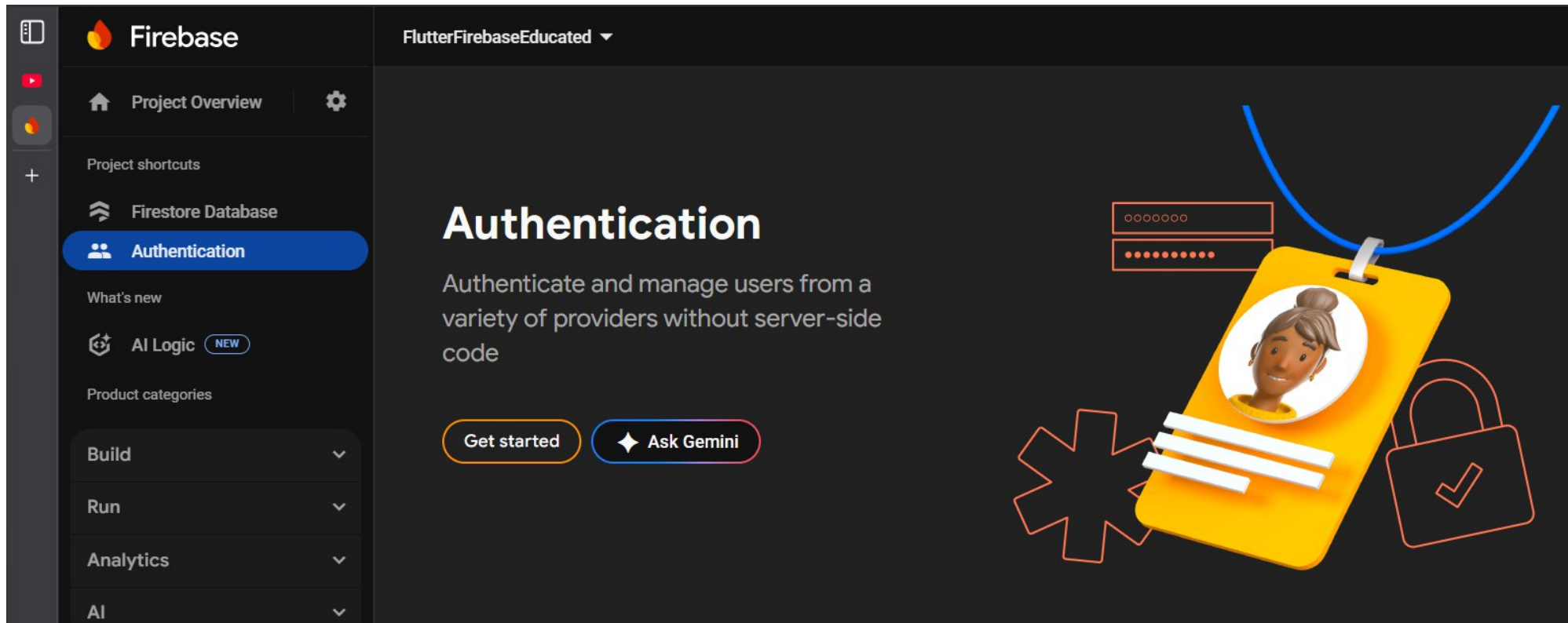
Authentication Method

- [Firebase Authentication](#) provides backend services. It supports authentication using passwords, phone numbers, as well as identity providers like Google, Facebook and Twitter, and more.

Native providers	Additional providers		
 Email/Password	 Google	 Facebook	 Play Games
 Phone	 Game Center	 Apple	 GitHub
 Anonymous	 Microsoft	 Twitter	 Yahoo

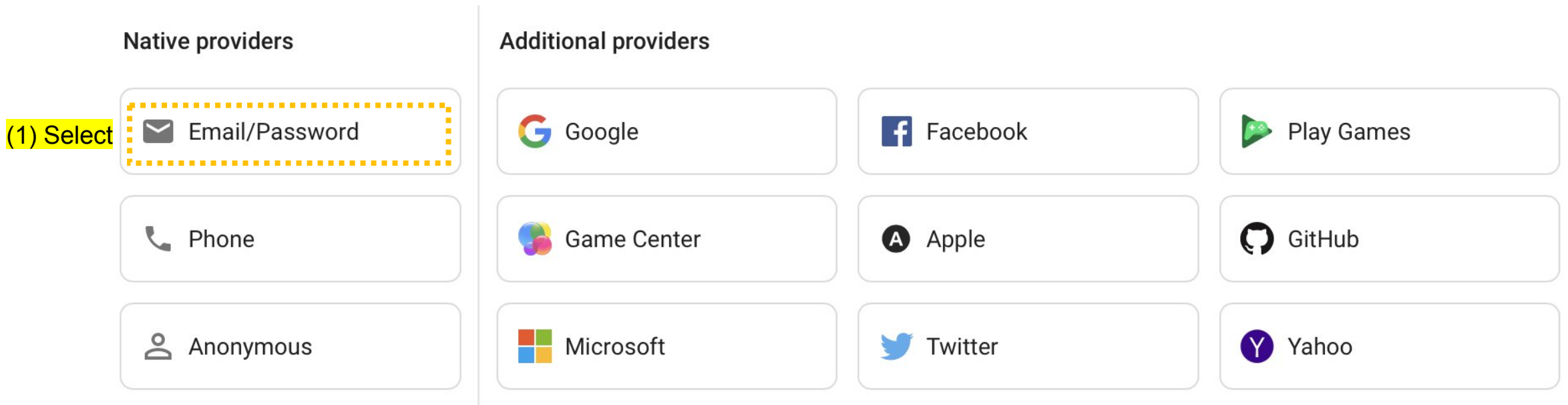
Firebase Authentication

- Go to Firebase console -> ไปที่ชื่อ Project ของตนเอง -> **Authentication**
- หากเข้ามาครั้งแรกและยังไม่เคยเปิดใช้งานมาก่อนจะพบหน้าจอเริ่มต้นดังนี้ ให้เลือกกด Get started



Firebase Authentication (cont.)

- เลือกไปยัง Sign-in Method
- ในตัวอย่างนี้จะขอใช้วิธีการ Authentication โดยใช้ Email/Password



Firebase Authentication (cont.)

- Sign-in Method : Native Provider - Email/Password

Sign-in providers

 Email/Password

☒ Enable

(1) Enable

Allow users to sign up using their email address and password. Our SDKs also provide email address verification, password recovery, and email address change primitives. [Learn more](#)

Email link (passwordless sign-in)

☐ Enable

Cancel

Save

(2) Save

Firebase Authentication (cont.)

- Go to Users tab and add new user

The screenshot displays the Firebase Authentication console for a project named 'Flutter-Educate'. The 'Authentication' section is active, with the 'Users' tab selected and highlighted by a dashed orange box. A yellow box with the text '(1) Select Users' is positioned over the 'Users' tab. In the top right corner, the text '(2) Add User' is visible. Below the tabs, a table of users is partially visible, with columns for Identifier, Providers, Created, Signed In, and User UID. A blue 'Add user' button, also highlighted by a dashed orange box, is located in the top right of the table area. A modal dialog titled 'Add an Email/Password user' is open in the foreground. It contains two input fields: 'Email' with the value 'example@mydomain.com' and 'Password' with the value '123456'. A yellow box with the text 'Example user' is placed below the email field. At the bottom of the dialog are 'Cancel' and 'Add user' buttons.

Flutter-Educate ▾

Authentication

Users Sign-in method Templates Usage

(1) Select Users

(2) Add User

Add user

Identifier	Providers	Created	Signed In	User UID
------------	-----------	---------	-----------	----------

Add an Email/Password user

Email Password

example@mydomain.com 123456

Example user

Cancel Add user

Change Rule of Database

- Modify rule of Firestore database and publish / [Read more](#)

Flutter-Educate ▾

Cloud Firestore

Data **Rules** Indexes Usage

Edit rules Monitor rules

Right now
unpublished changes

Today • 2:12 PM

Feb 25, 2022 • 9:38 AM

Feb 9, 2022 • 9:32 PM

unpublished changes | Publish Discard

```
1 rules_version = '2';
2 service cloud.firestore {
3   match /databases/{database}/documents {
4     match /{document=**} {
5       allow read, write: if request.auth.uid != null;
6     }
7   }
8 }
9 }
```

สร้างคลาสเพื่อรองรับการ ทำ Authentication

- สร้างไฟล์ภายใน lib/services ชื่อว่า `authentication_service.dart`
- คลาสชื่อว่า `AuthenticationService` มีรายละเอียดดังนี้

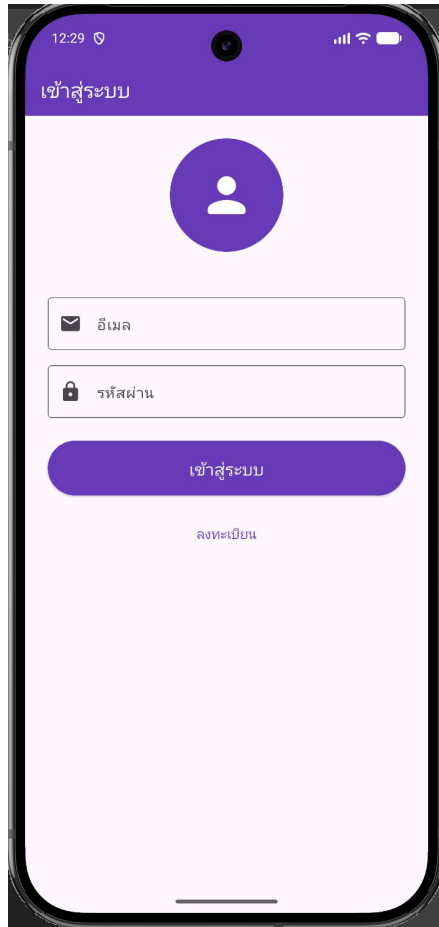
ทั้ง 2 ขั้นตอนนี้
ยังไม่ได้ดำเนินการ
อย่างสมบูรณ์



```
1 import 'package:firebase_auth/firebase_auth.dart';
2
3 class AuthenticationService {
4   final FirebaseAuth _auth = FirebaseAuth.instance;
5
6   Future<bool> login(String username, String password) async {
7     try {
8       UserCredential userCredential = await _auth.signInWithEmailAndPassword(
9         email: username,
10        password: password,
11      );
12       return userCredential.user != null;
13     } catch (e) {
14       return false;
15     }
16   }
17
18   Future<bool> register(String username, String password) async {
19     try {
20       final userCredential = await _auth.createUserWithEmailAndPassword(
21         email: username,
22         password: password,
23       );
24       return userCredential.user != null;
25     } catch (e) {
26       return false;
27     }
28   }
29
30   Future<void> logout() async {
31     // Implement logout logic here
32     // clear user session
33     await _auth.signOut();
34     // Clear any stored data on Local Storage or Shared Preferences if needed
35   }
36
37   bool isAuthenticated() {
38     // Check if the user is authenticated
39     // by checking if the current user is not null
40     return _auth.currentUser != null;
41   }
42 }
```

สร้างหน้าจอสำหรับการ Login

- สร้างไฟล์ภายใน lib/pages ชื่อว่า **login_page.dart**



```
// ปุ่ม Login
PreferredSize(
  width: double.infinity,
  child: ElevatedButton(
    onPressed: () {
      AuthenticationService().login(email, password).then((
        success,
      ) {
        if (success) {
          if (context.mounted) {
            Navigator.of(context).pushReplacement(
              MaterialPageRoute(
                builder: (context) =>
                  const HomePage(title: 'Electricity Payment'),
              ), // MaterialPageRoute
            );
          }
        } else {
          if (context.mounted) {
            ScaffoldMessenger.of(context).showSnackBar(
              const SnackBar(content: Text('เข้าสู่ระบบล้มเหลว')),
            );
          }
        }
      });
    },
    style: ElevatedButton.styleFrom(
      backgroundColor: Colors.deepPurple,
      foregroundColor: Colors.white,
      padding: const EdgeInsets.symmetric(vertical: 16),
    ),
    child: const Text(
      'เข้าสู่ระบบ',
      style: TextStyle(fontSize: 18, color: Colors.white),
    ), // Text
  ), // ElevatedButton
), // PreferredSize
```

```
import 'package:flutter/material.dart';
import '../services/authentication_service.dart';
import 'home_page.dart';

class LoginPage extends StatelessWidget {
  const LoginPage({super.key});

  @override
  Widget build(BuildContext context) {
    String email = '';
    String password = '';

    return Scaffold(
      appBar: AppBar(
        title: const Text('เข้าสู่ระบบ'),
        backgroundColor: Colors.deepPurple,
        foregroundColor: Colors.white,
      ), // AppBar
      body: SingleChildScrollView(
        child: Padding(
          padding: const EdgeInsets.all(24.0),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: <Widget>[
              > const CircleAvatar( // CircleAvatar ...
              > const SizedBox(height: 48),
              > // ช่องกรอกอีเมล
              > TextFormField( // TextFormField ...
              > const SizedBox(height: 16),
              > // ช่องกรอกรหัสผ่าน
              > TextFormField( // TextFormField ...
              > const SizedBox(height: 24),
              > // ปุ่ม Login
              > SizedBox( // SizedBox ...
              > const SizedBox(height: 16),
              > // ปุ่ม Register
              > TextButton( // TextButton ...
              > ], // <Widget>[]
            ), // Column
          ), // Padding
        ), // SingleChildScrollView
      ); // Scaffold
    }
  }
}
```

แก้ไขเพิ่มเติม main.dart

- แก้ไขในส่วนของ main ให้เรียกใช้ login page เป็นลำดับแรก

```
16  @override
17  Widget build(BuildContext context) {
18    return MaterialApp(
19      debugShowCheckedModeBanner: false,
20      theme: ThemeData(
21        useMaterial3: true,
22        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
23        bottomAppBarTheme: BottomAppBarThemeData(
24          color: Colors.deepPurple,
25        ), // BottomAppBarThemeData
26      ), // ThemeData
27      home: LoginPage(),
28    ); // MaterialApp
29  }
30 }
```

แก้ไขเพิ่มเติม home_page.dart

- เพิ่มเติม icon สำหรับ logout ที่หน้าจอ home

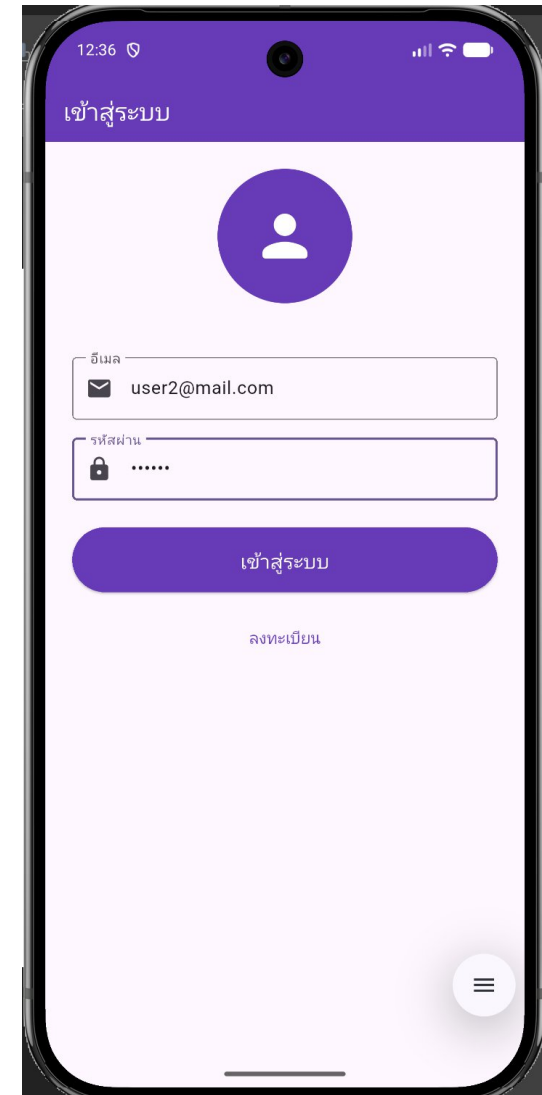


```
19 class _HomePageState extends State<HomePage> {
20   List<BillingRecordModel> billingItems = [];
21
22   @override
23   Widget build(BuildContext context) {
24     return Scaffold(
25       appBar: AppBar(
26         actions: [
27           IconButton(
28             icon: const Icon(Icons.logout),
29             onPressed: () {
30               // Implement logout logic here
31               // Clear user session and navigate back to login page
32               AuthenticationService().logout().then((_) {
33                 if (context.mounted) {
34                   Navigator.of(context).pushAndRemoveUntil(
35                     MaterialPageRoute(builder: (context) => const LoginPage()),
36                     (route) => false,
37                   );
38                 }
39               });
40             },
41           ), // IconButton
42         ],
43         title: Center(
44           child: Text(widget.title, style: TextStyle(fontSize: 18)),
45         ), // Center
46         backgroundColor: Colors.deepPurple,
47         foregroundColor: Colors.white,
48       ), // AppBar
```


การทดสอบ (การลงทะเบียน)

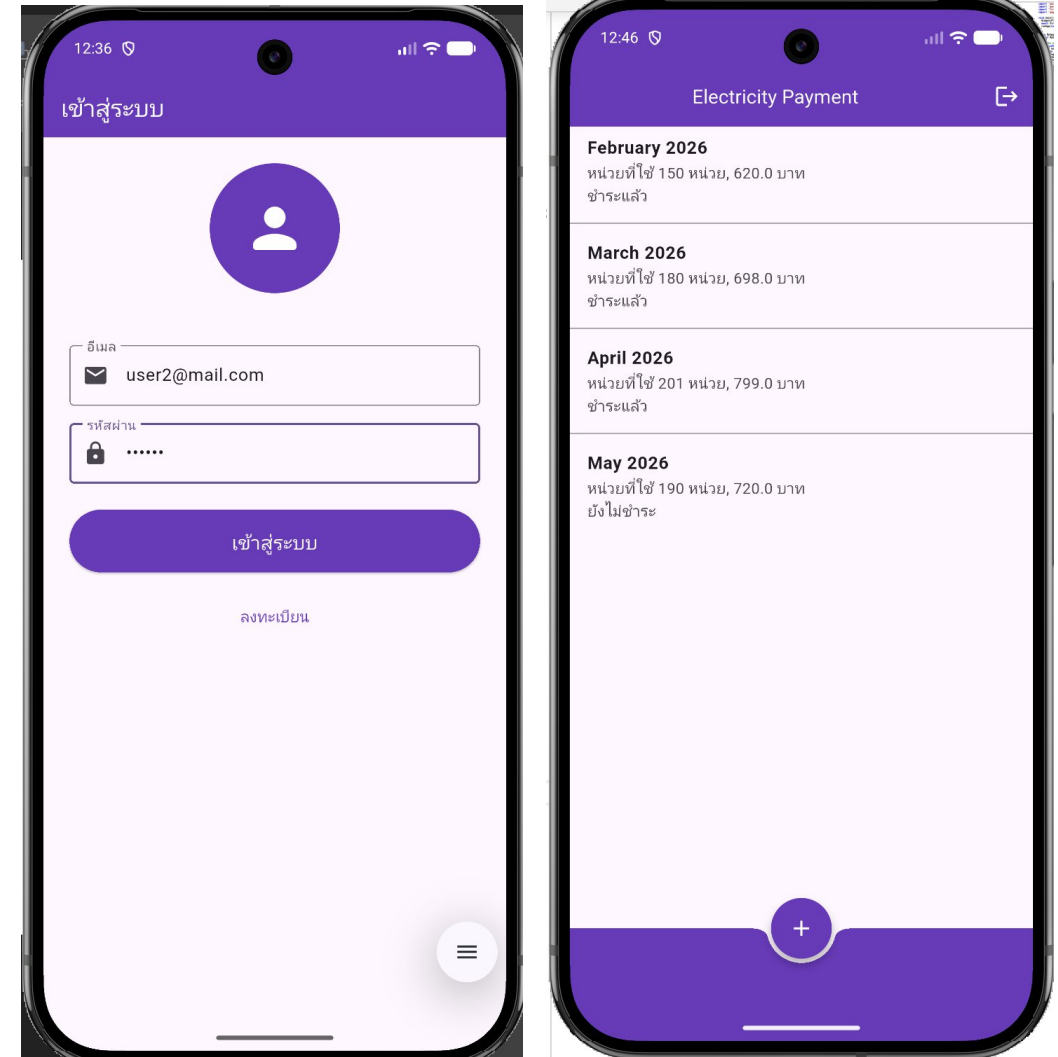
- การลงทะเบียน Email/Password
- ใส่ email และ password ที่ต้องการลงทะเบียนภายในกล่องข้อความ
- จากนั้นให้กดที่คำว่า ลงทะเบียน ด้านล่างปุ่มเข้าสู่ระบบ
- หากลงทะเบียนสำเร็จจะปรากฏแถบข้อความแจ้งผลขึ้นด้านล่าง และเมื่อ refresh หน้าเพจของ Firebase จะพบข้อมูล email ที่ลงทะเบียนปรากฏขึ้น

Search by email address, phone number, or user UID					Add user	↺	⋮
Identifier	Providers	Created ↓	Signed In	User UID			
newuser@mail.com	✉	Sep 9, 2025	Sep 9, 2025	5iDPlqkpt1SPSd5xILG4oWLOT...			
user1@mail.com	✉	Aug 15, 2025	Sep 1, 2025	szuc3QVZBsPBZLnuSivDgXvt...			
example@mydomain.c...	✉	Aug 15, 2025	Aug 15, 2025	duhGVyDFlibG4i4pSvkOEL3UB...			
					Rows per page:	50 ▼	1 – 3 of 3 < >



การทดสอบ (การ login)

- การเข้าสู่ระบบด้วย Email/Password
- ใส่ email และ password ที่ลงทะเบียนไว้ก่อนหน้านี้ภายในกล่องข้อความ
- จากนั้นให้กดที่คำว่า เข้าสู่ระบบ หากใส่ข้อมูลถูกต้องจะเข้าสู่หน้าจอ Home และแสดงข้อมูลที่บันทึกไว้บน Cloud Firestore



Q/A